TECHNOLOGICAL UNIVERSITY DUBLIN

MASTERS THESIS

Enhancing Static Malware Analysis with Large Language Models and Retrieval-Augmented Generation

Author:
Andre M M FARIA

Supervisor:
Dr Robert G SMITH

A thesis submitted in fulfillment of the requirements
for the degree of M.Sc
in Applied Cybersecurity

in the

School of Informatics and Cyber Security

May 2025

Declaration of Authorship

I, Andre M M FARIA, declare that this thesis titled, "Enhancing Static Malware Analysis with Large Language Models and Retrieval-Augmented Generation" and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this Institute of Technology Blanchardstown.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed:

Date: July 31, 2025

"The true masters are the friends we make along the way."

Anonymous

TECHNOLOGICAL UNIVERSITY DUBLIN

Abstract

School of Informatics and Cyber Security

M.Sc

Enhancing Static Malware Analysis with Large Language Models and Retrieval-Augmented Generation

by Andre M M FARIA

Malware's quick growth presents serious cybersecurity concerns, necessitating ongoing innovation in methods for detection, analysis, and mitigation. When it comes to handling complex and adaptable threats, traditional malware analysis techniques, which mostly rely on manual labour and rule-based systems, are progressively less effective. Large language models (LLMs) are a revolutionary way to automate malware analysis, even though Artificial Intelligence (AI) approaches have been effectively incorporated into cybersecurity. Current AI solutions, including machine learning classifiers and clustering algorithms, concentrate on aspects of malware but frequently lack the overall skills necessary to manage intricate datasets or fully understand virus behaviors.

This study suggests using LLMs to improve and automate static malware analysis. It seeks to automate crucial components of malware reverse engineering, such as code analysis, possible behavior interpretation and anomaly detection, by leveraging LLMs' capacity to handle and synthesize massive, heterogeneous information. In addition to addressing the drawbacks of manual and conventional AI techniques, this strategy adds scalability and adaptability to quickly changing malware threats.

In order to optimize the efficacy of malware analysis in cybersecurity, this study proposes a methodology that combines domain-specific datasets, such as malware sample databases, with prompt engineering techniques to evaluate samples and identify parameters such as malware classification categories and behavioral patterns on decompiled code. This approach is expected to provide a more comprehensive and accurate analysis of malware samples, as well as to improve the overall efficiency of malware analysis in cybersecurity.

Keywords: Malware Analysis, Large Language Models, Static Analysis, Cybersecurity, Artificial Intelligence, Generative Artificial Intelligence, Machine Learning, Reverse Engineering, Anomaly Detection, Code Analysis.

Acknowledgements

Thanks to my parents for supporting me through everything over the years…

Thanks to my supervisor, Dr Robert G Smith, for his guidance and support. :D

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Ever since the creation of the first computer by Charles Babbage in the 19th century, computers have been evolving at a rapid pace performing more and more tasks as the time passes. With this evolution, malicious actors began to notice manners in which to subvert established systems. These individuals, by force of belief or personal gain, have caused losses in the trillions to organisations and individuals across the globe.

As a response to these threats, these organisations and individuals began to develop techniques and tools to counter malicious actors. Some of these techniques revolve around the analysis of what software adversaries develop. The analysis of such malicious software is important in order to study and understand how they operate and the vulnerabilities they exploit to weaken, or outright topple, established security systems.

This research presents a the development of a methodology to analyse malicious software using artificial intelligence (AI) tools to enhance analysis quality and speed. This tools will be discusses toughly on 3.

This thesis presents a research about, and creation of, a method where RAG powered LLMs are introduced onto to the malware static analysis workflow in order to increase efficiency on the analysis.

1.1 Background and Motivation

1.1.1 Static Malware Analysis in Cybersecurity

These software artifacts created by malicious actors are called Malware. Static malware analysis focus specifically on techniques that decompose, or decompile, target Malware down to its most basic bytecode elements. These are then scrutinized to uncover malicious intent. This method of inspecting a program's structure, instructions, and information without actually running it, is one of the most popular and dependable tools in the cybersecurity toolbox. It is essential in settings where safety, speed, and scalability are critical since it eliminates the possibility of running potentially dangerous code. It is the keystone in early malware identification and categorization, due to its automation capabilities (i.e. antivirus engines or intrusion detection systems).

However, as threat actors evolve their tactics and adopt increasingly sophisticated evasion techniques, static analysis has begun to show its limitations. Many contemporary malware variants leverage obfuscation, packing, encryption, and code polymorphism to mask their functionality and frustrate traditional static inspection. Moreover, extracting actionable intelligence from static features often requires expert-level knowledge in assembly language, file formats, and API behaviour—creating a steep barrier for automation. These challenges have shown

the need for intelligent, context-aware tools capable of shortening the gap between low-level code patterns and high-level semantic understanding.

1.1.2 Emergence of LLMs in Code and Text Understanding

Over the past few years, Large Language Models (LLM) have rapidly transitioned from experimental research tools to foundational components across a wide range of natural language processing and code analysis applications. Built on transformer-based architectures and trained on vast corpora comprising not only natural language text but also programming languages and technical documentation, these models have demonstrated an impressive capacity to generate, summarize, translate, and reason about code and human language alike. Their success in handling structured and semi-structured data has opened new avenues in fields such as automated code completion, bug detection, documentation generation, and even code synthesis. With models like GPT, CodeBERT, and StarCoder pushing the limits of scale and capability, LLMs are starting to play a major part in determining how we interact with intricate codebases and interpret syntactic patterns that once required domain-specific knowledge.

This increased capacity in both textual and programmatic reasoning has generated interest in bringing LLMs to cybersecurity, where duties such as code interpretation, vulnerability identification, and malware classification have historically been left for highly trained analysts. Unlike rule-based systems, which rely on predetermined signatures or static heuristics, LLMs may generalise from a variety of examples and adapt to novel or obfuscated code patterns, which is very valuable in malware research. Furthermore, their capacity for natural language explanation enables a level of interpretability that aligns well with security workflows that demand transparency and traceability. While LLMs have shown early promise in tasks such as vulnerability detection and reverse engineering assistance, their full potential in enhancing static malware analysis, especially through integration with contextual retrieval systems, remains a largely untapped area of research, presenting an opportunity to reimagine traditional security paradigms with the help of advanced, language-aware intelligence.

1.1.3 Motivation

The reasoning behind the focus of this research is that static analysis remains to be a critical component of malware detection and neutralization. Security analysts are often required to analyze and process huge amounts of obfuscated data. By integrating RAG powered LLMs to this process we can make the analysis easier by tapping onto contextual knowledge from the RAG and the analysis power of LLMs.

1.1.4 Key Concepts

Here are defined all of the concepts and/or tools that are mentioned.

1.1.4.1 Large Language Models

Large Language Models (LLMs) are a type of multi-parameter Machine Learning (ML) algorithm designed for natural language processing. These models that are trained on a large volume of text using a multitude of learning techniques such as supervised and self-supervised learning.

Generative pre-trained transformers (GPTs) are the biggest and most powerful LLMs. Current models can be adapted to certain tasks (via fine-tuning) or directed by prompt engineering [[**brown2020languagemodelsfewshotlearners**]]. These models learn to predict the syntax, semantics and ontologies found in human language, but they also pick up biases and errors from the training data [[**10.1162/daed_a_01905**]]. Popular examples of LLMs include:

- GPT models (like GPT-4 or ChatGPT)
- Google's PaLM and Gemini
- Meta Llama series models

1.1.4.2 Prompt Engineering

Interactions with LLMs are made through *prompts*. Those are natural language text segments that describe the task the AI should perform [[**radford2019language**]]. They can be queries, commands or statements including context, instructions and/or relevant information.

Prompt engineering is the action of manipulating these text segments in order to optimize the wanted output [[**wahle-etal-2024-paraphrase**]]. For example, providing a prompt such as "What is an apple?" we have this output:

> An apple is a round, edible fruit produced by the apple tree (Malus domestica). It typically has smooth skin that can be red, green, or yellow, and a sweet to tart taste depending on the variety. Apples have a firm white flesh and contain a core with small black or brown seeds.
> They're commonly eaten raw, but also used in cooking and baking-like in apple pies, sauces, and juices. Apples are rich in fiber, vitamin C, and antioxidants, making them a popular choice for a healthy snack. The saying "an apple a day keeps the doctor away" hints at their reputation for promoting good health.
> Want to know about their origin, symbolism, or maybe how they're grown?

This output is acceptable, however it can be modified with some prompt engineering, such as an ask to change the output formatting. Then, providing the phrase "Provide only bullet points." in addition ot the original we have:

> * Edible fruit from the apple tree (Malus domestica)
> * Typically round with red, green, or yellow skin
> * White, crisp flesh with a sweet to tart flavor
> * Contains a core with small seeds
> * Commonly eaten raw or used in cooking and baking
> * Rich in fiber, vitamin C, and antioxidants
> * Grown in temperate climates worldwide
> * Symbol of health, knowledge, and temptation in various cultures

1.1.4.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [[**lewis2021retrievalaugmentedgenerationknowledgeintensiven**
is a technique that allows for LLMs to query information from a specific type of datastore and use it as context for the response it provides. This allows for adaptive

learning of the models without the need of retraining (fine-tuning). It improves the quality of LLM output as the model does not rely solely on pre-trained knowledge but also on relevant information from the datastore.

RAG works by having the LLM to query the datastore using vector search, embeddings or keyword matching (depending on the datastore type), feeding this data onto the LLM input and then continuing with the normal LLM process for output generation.

1.1.4.4 Malware

Short for *Malicious Software*, Malware is any program or code that is created for malicious purposes like exploitation of computer systems networks or users. Often created with monetary gain in mind, these programs activities include stealing sensitive data, damage systems, disrupt operations or gain unauthorized access to systems or environments. Common types of malware include:

- Viruses: Attach themselves to legitimate software, spreading when the software is run.
- Worms: Autonomous malware capable of multiplying across networks without needing human intervention. Exploit network weaknesses to infiltrate other systems without permission.
- Trojans: Also known as Trojan Horses, these are disguised as genuine software to trick users into running them.
- Ransomware: Encrypts or locks files, demanding payment (ransom) for restoration.
- Spyware: Stealthily gathers sensitive data and user activities without the user's consent.
- Adware: Delivers unwanted advertisements, potentially slowing down or compromising systems.

1.1.4.5 Malware Analysis

As mentioned before, the software created by malicious actors needs to be evaluated and analyzed to understand how it works, what exploits it takes advantage of and how these exploits can be patched. As a basis, there are two types of analysis of software that can be made to understand any kind of software. These are as follows:

- **Static Analysis**: Focuses on looking at the sequences of individual instructions on the software bytecode to identify the its characteristics, such as identification of the parameters in which the analyzed software is built upon (e.g. Operating system it executes on, processor architecture, etc), what execution paths it employs, what system calls it performs, etc. This analysis is done without running the code and it requires a lot of time and effort from the analyst.
- **Dynamic Analysis**: Entails running the potential malicious software to gather runtime information such as network calls and system call execution data which are not visible through static analysis. This analysis is done by creating a special (sandbox) environment (i.e docker container and/or a virtual machine) and running the software.

1.1.4.6 Static Analysis vs Dynamic Analysis

Both static and dynamic analysis have their own benefits and are important on their own right for understanding how malware is designed and developed. While static analysis takes advantage of its speed and safety where files do not need to be executed to be analyzed, dynamic analysis enables a more extensive analysis which might not be visible with static analysis alone.

However, both have their limitations such as the increased computational cost and requirement of a secure environment of dynamic analysis versus the vulnerability of static analysis to obfuscation and encryption techniques.

1.2 Problem Statement

TODO...

1.3 Research Aim and Objectives

1.3.1 Aim

Research the enhancement of static malware analysis using RAG-enhanced LLMs

1.3.2 Objectives

- Develop a method for integrating LLMs with RAG to enhance static malware analysis.
- Evaluate the effectiveness of RAG-enhanced LLMs in identifying, explaining, and comparing malware threats using static features (e.g., file structure, bytecode, API calls) against traditional static malware analysis techniques in terms of accuracy, efficiency, and interpretability, while identifying challenges and potential risks (e.g., adversarial manipulation and hallucination).

1.4 Research Questions and Hypothesis

1.4.1 Questions

These are the research questions that will guide the development of this thesis.

- How LLMs can be leveraged to enhance static malware analysis?
- How does a RAG-enhanced LLM compare to traditional static analysis techniques in terms of accuracy, efficiency, and interpretability?
- What role does RAG play in improving LLM-driven malware classification, particularly in terms of contextual relevance and justifiability of results?

1.4.2 Hypothesis

- Hypothesis 1: LLMs can enhance static malware analysis by accurately interpreting and classifying malware samples based on static features such as bytecode, file structure, and API calls, offering greater adaptability and insight than rule-based static tools.

- Hypothesis 2: A RAG-enhanced LLM will outperform traditional static analysis techniques in terms of interpretability and contextual understanding, while maintaining comparable levels of accuracy and efficiency in malware classification.
- Hypothesis 3: RAG improves LLM-driven malware classification by providing relevant contextual information during inference, leading to more justifiable and semantically grounded explanations of classification results.

1.5 Methodology Overview

TODO...

1.6 Contributions of the Research

TODO...

1.7 Thesis Structure

TODO...

Chapter 2

Literature Review

This chapter presents the current state of the art on static malware analysis, with a focus on the integration of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) to enhance automation, semantic understanding, and explainability in malware detection workflows. Static analysis plays a crucial role in cybersecurity, enabling the examination of malicious code without execution. Its safety, repeatability, and efficiency make it essential for malware triage and reverse engineering tasks [4]. However, challenges such as code obfuscation, semantic ambiguity, and limited contextual awareness continue to limit its effectiveness [7, 2].

Traditional approaches, such as signature-based and heuristic techniques, have been widely deployed in antivirus engines. These methods depend on predefined rules and known byte patterns, which are easy to circumvent with packing or polymorphic transformations [12]. In response to these limitations, Machine Learning (ML) and deep learning models have been introduced to detect and classify malware based on statistical features and behavior signatures extracted from static data [2, 7]. Despite notable improvements in detection rates, many of these models require extensive feature engineering, struggle with generalization, and lack transparency regarding decision-making processes.

Recently, the emergence of large-scale language models such as GPT-4 has opened new avenues for augmenting static malware analysis. Fujii and Yamagishi [6] demonstrated that LLMs can explain decompiled functions in plain language and provide analysts with faster contextual insights, particularly when guided with structured prompts or roles. These findings support the use of Model-Centric Prompting (MCP) as a methodology for optimizing prompt engineering in code-focused tasks [13]. However, their work also exposed limitations in accuracy and hallucination when insufficient context is provided.

RAG offers a promising solution to these challenges by dynamically incorporating external information—such as threat reports or similar malware case studies—into the prompt context [8]. While RAG has shown success in NLP applications like question answering and summarization, its potential in malware analysis remains underexplored, representing a key research opportunity identified in this study [9].

This literature review synthesizes current efforts across five key themes:

- Fundamentals and mechanisms of LLMs and RAG.
- Characteristics of malware and the importance of static analysis.
- Applications and limitations of traditional and ML-based static malware analysis.
- Existing and emerging roles of LLMs in cybersecurity.

- Opportunities for integrating RAG and LLMs to enhance analysis accuracy and explainability.

2.1 Static malware Analysis

Static malware analysis refers to the examination of malicious software without executing it, typically by inspecting its binary code, structure, metadata, and associated artifacts. It remains a foundational practice in cybersecurity due to its safety, speed, and reproducibility [4, 7, 2]. Analysts use this method to infer program functionality, extract Indicators Of Compromise (IOCs), and generate initial threat intelligence without the risk of executing potentially harmful code.

Basic static analysis methods include file header inspection (e.g., PE header metadata), string extraction, and checksum analysis. Advanced techniques rely on disassembly, Decompilation, and control flow graph analysis to reconstruct higher-level logic [4, 15]. These techniques provide insights into what the malware does and how it operates, but suffer from challenges such as obfuscation, code packing, and the absence of runtime context.

Despite their limitations, static methods offer advantages over dynamic analysis, especially for rapid triage and offline forensic evaluation. They are immune to anti-virtualization and anti-debugging tactics often used by sophisticated malware. However, modern threats often employ techniques like runtime linking, encrypted payloads, and evasion frameworks, limiting the reach of purely static tools [4, 15].

These limitations have created interest in augmenting static analysis with intelligent reasoning systems, especially LLMs, which are explored in the following sections.

2.1.1 Signature-Based and Heuristic Methods

Signature-based analysis matches specific byte patterns, string literals, or opcode sequences in a binary against known malware fingerprints. This method is fast and widely used by analysis tools such as *ClamAV*, *YARA*, and *VirusTotal* [4]. However, even slight changes to the malware's structure—such as packing or encryption—can render signatures ineffective.

Heuristic analysis improves on this by flagging suspicious patterns based on known behaviors or structural anomalies [12]. For example, unusual import tables, abnormal section sizes, or the use of specific API calls (e.g., `VirtualAlloc`, `CreateRemoteThread`) are often considered red flags. Yet these rule-based systems are limited by their brittleness and potential for false positives [7, 2].

2.1.2 Machine Learning-Based Approaches

To overcome the rigidity of traditional methods, static malware analysis has increasingly incorporated ML [2]. These approaches use feature representations—such as n-grams, opcode frequencies, or entropy values—to train classifiers capable of detecting both known and unknown malware samples. Datasets like EMBER, Malimg, and VirusShare facilitate these developments by providing labeled corpora for training and evaluation [7, 1].

However, challenges remain. Feature engineering often requires expert domain knowledge, and deep learning models can be opaque or hard to interpret. More importantly, many ML-based detectors struggle with obfuscated or packed malware where feature extraction fails to surface meaningful signals [2].

2.1.3 Practical Considerations and Analyst Workflows

According to field studies such as Wong et al. [15], malware analysts adopt a tiered approach to static analysis, starting with basic tools like hash extractors and moving toward disassembly and Decompilation as needed. Tools like `Ghidra`, `IDA Pro`, and `Radare2` are central to this process, offering powerful Decompilation and navigation features, albeit requiring significant expertise.

Fedák and Štulrajter [4] provide detailed insight into the practical toolkit of an analyst, listing tools such as:

- **Strings**, **HexDive**, and **BinText** for readable text extraction.
- **PEiD** and **Exeinfo PE** for identifying compression techniques and signatures.
- **CFF Explorer** and **Dependency Walker** for PE header inspection and DLL analysis.

Static analysis of PE (Portable Executable) files—commonly used by Windows binaries—reveals critical metadata such as entry points, import address tables, and linked libraries. This data provides early insights into malware behavior, even before Decompilation [4].

2.1.4 Obfuscation, Compression, and Runtime Challenges

Modern malware often leverages runtime linking and obfuscation to bypass static detection. Code is compressed or encrypted, with only loader stubs visible to analysts. These stubs typically use runtime API resolution (e.g., via `LoadLibrary`, `GetProcAddress`) to dynamically fetch and execute malicious payloads [4].

The prevalence of obfuscation significantly reduces the efficacy of basic static tools. For example, string extraction becomes nearly useless if meaningful strings are encrypted. In such cases, unpacking and de-obfuscation—either manual or using tools like Exeinfo PE—are essential to enable deeper analysis [15].

2.1.5 Summary of Limitations and Opportunities

While static malware analysis remains a cornerstone of threat detection, its limitations are well-documented [15, 4]:

- Lack of semantic understanding (e.g., what the malware does or why it behaves that way).
- High dependency on manual expertise for effective use of tools.
- Vulnerability to evasion through obfuscation, packing, and runtime behavior.

These limitations have motivated the exploration of Artificial Intelligence (AI) driven approaches that promise greater automation, semantic reasoning, and contextual interpretation.

2.2 The Role of Language Models in Cybersecurity

LLMs have rapidly become key enablers in cybersecurity, offering powerful automation for tasks such as malware detection, vulnerability analysis, code auditing, incident summarization, and threat intelligence extraction. Pre-trained

on diverse code and text, LLMs like `GPT`, `BERT`, `CodeBERT` [3] and `Codex` show impressive generalization and reasoning over both structured and unstructured data [11, 13, 5, 10].These models are built upon the Transformer architecture, introduced by Vaswani et al. [14], which uses self-attention mechanisms to model global dependencies efficiently and supports highly parallelizable training.

In the domain of malware analysis, LLMs are particularly useful for interpreting decompiled or disassembled functions and generating natural-language summaries. Fujii and Yamagishi [6] demonstrated that GPT-4 can accurately explain functions when prompted appropriately—especially using role-based prompts such as "You are a malware analyst. . . ." This highlights the importance of Model-Centric Prompting (MCP), where prompts are tailored to align with model reasoning behavior rather than merely input format [13].

Recent surveys such as Nourmohammadzadeh et al. [11] and Ferrag et al. [5] show growing interest in applying LLMs to cybersecurity, but also emphasize that these models require adaptation to operate on binary code, disassembled structures, and malicious behavior patterns. The research by Metta et al. [10] provides a comprehensive risk-centric perspective, highlighting both defensive and offensive uses of LLMs—from malware analysis and reverse engineering to code generation for adversarial purposes.

Key benefits of LLMs in cybersecurity include:

- **Automated explanation and triage:** translating complex code into natural language.
- **IOC and pattern extraction:** identifying suspicious API calls, encoded payloads, or unusual imports.
- **Augmenting human analysts:** supporting SOC workflows through summarization and prioritization.

However, several limitations persist:

- **Hallucination and trust:** LLMs may confidently produce incorrect or fabricated information, especially without sufficient context.
- **Prompt sensitivity:** output quality varies based on structure and phrasing [13].
- **Black-box reasoning:** difficulty auditing or interpreting why a model made a specific decision [10].
- **Security concerns:** models may be vulnerable to prompt injection, adversarial inputs, or training data poisoning.
- **Opacity:** The internal reasoning of LLMs remains a "black box," limiting trust and auditability [11].

Ferrag et al. (2024) further explore the vulnerabilities and risks associated with deploying LLMs in cybersecurity workflows, including prompt injection, data poisoning, and adversarial manipulation—even as they highlight methods for adaptation like fine-tuning, retrieval augmentation, and reinforcement learning [5].

Despite these challenges, LLMs contribute meaningfully to cybersecurity by reducing cognitive load and accelerating analysis [5]. When combined with techniques like RAG, their outputs can be better grounded, more reliable, and transparently justified.

2.3 RAG in NLP

RAG is a technique that enhances the performance and factuality of LLMs by supplementing them with external, retrievable knowledge sources. Instead of relying solely on pre-trained parameters, RAG models incorporate a retrieval module that fetches relevant context—such as documents, reports, or structured data—which is then used to condition the generative response. This paradigm is particularly beneficial in knowledge-intensive tasks such as question answering, summarization, and factual reasoning [9, 8].

Lewis et al. [9] introduced the original RAG framework by combining a dense retriever with a seq2seq LLM, demonstrating improvements in open-domain QA by grounding outputs in retrieved passages. Subsequent research has refined the retrieval mechanisms (e.g., dense vs. sparse, hybrid models), document chunking strategies, and integration techniques, leading to significant gains in factual correctness and reduced hallucinations across tasks.

Gao et al. [8] provide a comprehensive survey of RAG systems and identify key benefits:

- **Factual accuracy:** Retrieved documents act as grounding references, reducing the likelihood of hallucinations.
- **Scalability:** Retrieval modules can be updated independently, avoiding the need to retrain the base model.
- **Context expansion:** RAG enables models to dynamically adapt to task-specific or domain-specific corpora.

These properties make RAG a compelling approach in domains where correctness, recency, and traceability of information are critical—such as medicine, law, and increasingly, cybersecurity.

In the context of malware analysis and static code reasoning, RAG holds significant potential: analysts often consult documentation, malware reports, or IOC databases during investigations. A RAG-augmented LLM could mimic this behavior by retrieving semantically relevant samples, threat profiles, or code analogues to provide better-informed and more explainable outputs.

The security of RAG-based systems is an emerging concern. Zou et al. [16] introduced the *PoisonedRAG* threat model, showing that adversaries can corrupt the retrieval index to manipulate model outputs via malicious documents. Their findings underscore the need for trusted, sanitized retrieval pipelines and integrity checks when applying RAG in high-stakes domains like cybersecurity.

Despite these risks, RAG remains a promising solution to many of the context limitations identified in static malware analysis. It enables LLMs to ground their outputs in verifiable sources, improve justifiability, and reduce hallucination—all of which align with the goals of this thesis. The next section explores the conceptual integration of RAG and LLMs for static malware analysis and highlights emerging research directions at this intersection.

2.4 Combining rag and LLMs for Static malware Analysis

While LLMs have demonstrated utility in malware analysis tasks such as code explanation, IOC extraction, and classification, they are often constrained by limited context windows and the absence of external knowledge. RAG offers a

natural extension to overcome these limitations by dynamically supplying relevant, grounded information during inference.

Static malware analysis workflows frequently rely on external resources—such as malware repositories, IOC databases, Decompilation guides, threat reports, and *YARA* rule libraries—to contextualize findings. A RAG-enhanced LLM can emulate this expert behavior by retrieving semantically similar samples, annotated analyses, or supporting documentation as prompt context [9, 8]. This contextual grounding enables the model to:

- Improve detection accuracy for obfuscated or novel malware by comparing with similar known variants.
- Enhance interpretability through grounded justifications drawn from previous samples or reports.
- Reduce hallucinations by anchoring generation in retrievable, verifiable content.

Although few studies have explicitly combined RAG with static malware analysis pipelines, early evidence in adjacent fields supports this integration. In source code understanding and vulnerability detection, retrieval-enhanced models improve performance by supplying relevant snippets or past issue reports to the model at inference time. This inspires analogous use in static malware contexts, where samples with similar bytecode structures or behavioral markers can be retrieved and used to interpret new or ambiguous binaries.

Moreover, Fujii and Yamagishi [6] highlighted that LLM hallucination increases when insufficient context is provided to the model, especially in malware-specific tasks. Integrating RAG directly addresses this issue by enriching the prompt with retrieved knowledge, which can include:

- Decompiled code from known malware families.
- Threat intelligence reports on similar IOC patterns.
- Descriptions of suspicious API usage from documentation.

Such retrievals not only make model outputs more reliable but also provide analysts with traceable reasoning paths. When paired with structured prompt design methodologies like Model-Centric Prompting (MCP), the combined RAG-LLM approach could significantly improve automation and accuracy in static analysis pipelines.

Security and reliability challenges remain. As noted by Zou et al. [16], poisoned documents or manipulated retrieval indices can lead to knowledge corruption and misleading outputs. To mitigate these risks, trusted retrieval indices, curated malware corpora, and retrieval integrity checks should be considered foundational elements of any cybersecurity-focused RAG pipeline.

The integration of RAG and LLMs into static malware analysis pipelines is an emerging and largely untapped area. Its benefits—contextual grounding, improved explainability, and reduced hallucination—are directly aligned with the needs of modern malware analysts.

2.5 Gaps in Current Literature

The reviewed literature illustrates steady progress in static malware analysis and the adoption of AI in cybersecurity. However, several significant gaps persist—both technical and methodological—that motivate this research.

2.5.1 Lack of Integration Between rag and malware Analysis

While RAG has demonstrated success in natural language tasks such as question answering and summarization [9, 8], its application in malware analysis—particularly static workflows—remains largely unexplored. No current work bridges LLM-driven reasoning with contextual retrieval of prior malware samples, reports, or IOC patterns, despite its strong theoretical fit.

2.5.2 LLM Use is Largely Exploratory and Unbenchmarked

LLM-based research in static analysis is still in its infancy. Studies like Fujii and Yamagishi [6] provide qualitative results demonstrating feasibility, but there is a lack of standardized benchmarks, evaluation frameworks, or comparative studies with traditional tools such as YARA or ML classifiers. The GenAI in Cybersecurity report [10] also notes the scarcity of domain-specific evaluation methods that account for explainability and analyst trust.

2.5.3 Explainability, Auditability, and Analyst Trust Are Underserved

Interpretability is essential for analysts making high-stakes security decisions [15]. However, most AI-driven malware analysis models—including LLMs—fail to provide transparent reasoning or traceable justifications. This issue is compounded by hallucinations and black-box behaviors, limiting practical adoption [10, 13].

2.5.4 Underutilization of Prompt Engineering and Interaction Protocols

Effective use of LLMs requires structured prompt design. Yet, most cybersecurity applications ignore prompt engineering methodologies like Model-Centric Prompting (MCP), which are shown to improve output reliability and task alignment [13]. A lack of formalized interaction strategies hinders reproducibility and evaluation.

2.5.5 Emerging Security Risks in AI-Augmented Pipelines

As outlined by Zou et al. [16] and echoed in the generative AI security report [10], RAG-based and LLM-powered systems face risks such as data poisoning, prompt injection, and misinformation amplification. None of the surveyed malware analysis frameworks account for these threats, indicating a serious oversight in current literature.

By addressing these gaps, this research seeks to develop a reproducible framework that combines RAG, LLMs, and structured prompts to enhance static malware analysis.

2.6 Summary

This chapter has surveyed the state of the art in static malware analysis, the emerging role of LLMs in cybersecurity, and the promising capabilities of RAG in improving contextual reasoning and output reliability.

We began by reviewing traditional and machine learning-based static analysis techniques, highlighting their advantages in safety and scalability but also their limitations—particularly in handling obfuscation, semantic interpretation, and novel threats [4, 7, 2]. Next, we explored how LLMs offer new possibilities for automating and interpreting malware analysis, while also facing challenges related to hallucination, prompt sensitivity, and explainability [6, 11].

RAG was introduced as a powerful strategy to address LLM limitations by supplying external context at inference time. Its benefits include improved factual grounding, reduced hallucination, and adaptability to evolving threat landscapes [9, 8]. However, its use in malware analysis remains nascent, and security concerns — such as retrieval poisoning — must be considered carefully in high-assurance settings [16].

Combining RAG with LLMs in the context of static malware analysis represents an underexplored research opportunity. The literature currently lacks comprehensive studies evaluating such integrations or establishing reproducible, explainable pipelines for cybersecurity tasks. This gap is further compounded by minimal usage of structured prompting methodologies like Model-Centric Prompting (MCP), which could improve the interpretability and robustness of AI-generated analyses [13].

The insights from this chapter establish the foundation for the proposed research. In the next chapter, a methodology is introduced to develop and evaluate a framework that combines RAG, LLMs, and structured prompts for static malware analysis — focusing on accuracy, explainability, and resilience.

Chapter 3

Methodology

This chapter outlines the methodology and experimentation applied for this thesis. The research adopts and experimental and comparative approach to examine how RAG-enabled large language models LLMs can improve static malware analysis. Due to the relative novelty of using RAG in this field, the work is both exploratory and experimental, involving the deployment and assessment of a functional prototype. In order to enable practical comparison with conventional static analysis techniques, it integrates a prototype-based implementation and assessment.

3.1 Research Design

The research will be focusing on two core analysis manners that will enable a direct comparison between the traditional static approach and the modern AI-assisted method. Those are:

- **Traditional Static Analysis**: Malware samples are analysed using established static analysis techniques, including feature extraction (e.g., strings, byte entropy, imported functions, PE structure) and signature, or heuristic, based classification. Tools such as PEFrame or existing VirusTotal metadata will serve as the baseline for labeling and behaviour inference.
- **LLM + RAG-Assisted Analysis**: A RAG pipeline is implemented using Haystack and a vector database (PostgreSQL with pgvector).Static features from malware samples are used to query a pre-built knowledge base containing labelled malware data, technical documentation, and behavioural summaries. Retrieved documents are injected into prompts and passed to a general-purpose LLM (e.g., GPT-4o orDeepSeek-R1), which is tasked with classifying the sample, explaining its reasoning, and providing relevant threat context.

By using this method,

3.1.1 Experimental Workflow

Each malware sample will undergo:

- **Feature Extraction**: Using tools or scripts to extract static features (e.g., strings, API calls, file structure).
- **Baseline Analysis**: Classification and interpretation using traditional techniques or metadata.

- **LLM + RAG Analysis**: Feature-based queries retrieve contextual documents, which are embedded into LLM prompts. The model then classifies the sample and provides reasoning.

Results from both configurations will be logged and evaluated using:

- **Quantitative metrics**: Accuracy, precision/recall, inference time.
- **Qualitative measures**: Interpretability, contextual relevance, and reliability of LLM outputs.

All prompt templates, retrieval settings, and evaluation criteria will be documented.

3.2 System Architecture

This section outlines the overview of the pipeline architecture. These steps have been designed to modularize the analysis, ensure reproducibility and allow for algorithmic changes whenever needed. Below, the diagram of the pipeline elements is displayed along with an overview on each of them.

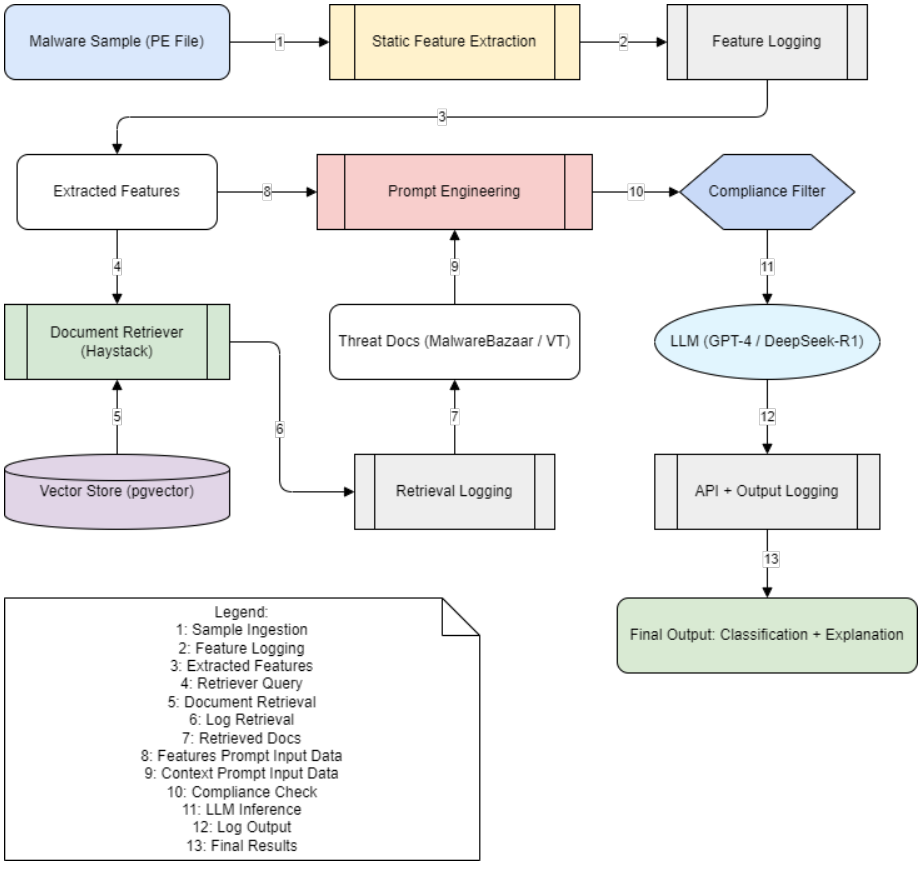

FIGURE 3.1: System Pipeline Diagram

1. **Malware Sample Ingestion:** Raw malware binaries (e.g., PE files) are sourced from public repositories such as MalwareBazaar or accessed via API from VirusTotal. Each file is stored with a unique identifier (e.g., SHA256 hash).

2. **Static Feature Extraction:** Using tools such as `pefile`, the system extracts static attributes from the malware, including:
    - PE header metadata
    - Imported API calls
    - Section structure and entropy
    - Printable strings
3. **Log Feature Extraction:** All extracted features are logged with timestamps, sample identifiers, and summary statistics.
4. **Feature-Based Retrieval Query:** The extracted static features are embedded into vector representations (if needed) and used as queries to a semantic search index.
5. **Document Retrieval:** Relevant documents (e.g., threat reports, malware family descriptions) are retrieved from an external datastore to be provided as context.
6. **Log Retrieval Step:** Retrieval activity is logged, including query terms, matching document IDs, retrieval time, and the data source used (e.g., VirusTotal vs. local data).
7. **Prompt Construction:** Retrieved documents and extracted features are formatted into structured prompts. Templates are used to ensure consistent formatting across LLM queries.
8. **Compliance Check Module:** Before sending prompts to the LLM, a filtering mechanism ensures that no disallowed content (e.g., malware payloads, sensitive data, proprietary code) is included.
9. **LLM Inference (GPT-4 / DeepSeek-R1):** The prompt is passed to a large language model (either GPT-4 or DeepSeek-R1) to generate:
    - Malware family classification
    - Explanation or justification of the classification
    - Comparative analysis against known malware types
10. **Log API Call:** Each API interaction is logged, including:
    - Model version and endpoint
    - Prompt metadata (not full text for privacy)
    - Token usage, latency, and return timestamp
11. **Output Generation:** The model output (classification + explanation) is presented to the user or stored for evaluation.
12. **Results Logging and Storage:** Final outputs, along with model metadata and hash references, are stored for downstream analysis or human review.

3.3 Data Sources

TODO...

3.4 Retrieval-Augmented Generation (RAG) Pipeline

TODO...

3.5 Prompt Engineering

TODO...

3.6 Toolchain

This section outlines the used tooling for the experiments and some discussion on why they were chosen.

3.6.1 Programming Language and Environment

TODO...

3.6.2 Language Models

TODO...

3.6.3 RAG Framework and Storage

TODO...

3.6.4 Malware Intelligence Sources

TODO...

3.6.5 Evaluation and Visualization

TODO...

3.7 Evaluation Framework

TODO...

3.8 Reliability, Validity, and Limitations

TODO...

3.9 Ethical Considerations

TODO...

3.10 Summary

TODO...

Chapter 4

Discussion of Results

TODO...

4.1 Summary of Key Findings

TODO...

4.2 Interpretation of Findings

TODO...

4.3 Implications of the Study

TODO...

4.4 Limitations

TODO...

4.5 Recommendations for Future Research

TODO...

4.6 Conclusion

TODO...

Chapter 5

Conclusion

TODO...

5.1 Summary of Research

TODO...

5.2 Answers to Research Questions

TODO...

5.3 Research Contributions

TODO...

5.4 Implications of the Study

TODO...

5.5 Limitations

TODO...

5.6 Recommendations for Future Research

TODO...

5.7 Final Reflections

TODO...

Appendix A

Frequently Asked Questions

A.1 Getting Feedback

1. Get feedback **often** and from different audiences – your family, friends, professors, colleagues, advisor, other graduate students. The more you talk about your research, the more comfortable you get with it.

2. Keep a positive attitude. Research is hard. If it were easy, everyone would be doing it.

3. Consider setting up or joining a thesis group to share your ideas and experiences.

Supervisor's feedback

Some supervisors will ask for you to send each chapter as you complete it, offering feedback at that point, and then again at the end when the thesis chapters are collated. Other supervisors may want to be more involved, and there are others who will not want to see your thesis until it is completed by your standards. Whichever approach your supervisor takes, be aware that they will need some time to read through your work and provide feedback. Your thesis review is likely not the only piece of work your supervisor is undertaking, so be patient and factor review time into your work schedule.

A couple of points to note about supervisor feedback:

- You will receive feedback on your approach to research (i.e., method, experiment design etc...) as well as your writing. It is your responsibility to take notes at meetings etc. in order to record this feedback. It is also up to you whether or not you act upon the feedback provided.

- Your supervisor's role is to guide your work. It is not their job to complete the research, suggest methods, design experiments, or to write/rewrite sections of your thesis.

- Your supervisor should be supportive but sometimes their feedback may be difficult to hear. Just remember, their goal is to guide you and to make you a better researcher. Learn to have your work criticised in a constructive manner, it is part of the learning process.

- It is not the role of your supervisor to proofread your thesis. Many supervisors will point out typos, grammatical errors or styling issues etc. when they see them, but this is not their role.

A.2 Proofreading/copyediting

It is important to have your work proofread[1]. If English is not your first language, this is even more important for you.

How?

A good approach is to proofread yourself as you write and then again when you are finished writing a section or chapter. When you have a near final draft, have it proofread by a friend, family member, colleague, or a classmate etc... (not your supervisor). Choose your proofreader wisely. Make sure that they have good written English skills and are able to spot grammatical errors. A native English speaker can be good for this but not all native English speakers have the skills needed to be a good proofreader.

There are many things to look out for when reviewing your own work, everything from text alignment and section numbering, to figures and tables, to spelling and grammar. It's best to identify and fix any of these errors immediately. Don't wait until the end because these will build up and it often takes longer than you think to fix them.

If you find that you make the same mistake regularly, e.g., you misspell the same word regularly, or you use a colon where you shouldn't, then make a list of these to check back when you are finished each section (the search feature is good for this).

[1]Two types of editing that are commonly used interchangeably are copy editing and proofreading. Both types of editing clean up writing, but each has its distinct contribution to the process. https://thesiswhisperer.com/2016/11/30/doing-a-copy-edit-of-your-thesis/

A.3 Writing Assistants

In the past, students may have used tools such as Grammarly or Quetext, but this has become more problematic because such editing tools now come with AI assisted writing (see more here: https://tudublin.libguides.com/c.php?g=720901&p=5233062).

Using tools such as a spell checker, a grammar checker, and a punctuation checker are generally acceptable. Using more advanced tools to rewrite sentences, check tone, offer alternative word choices, offer citations etc... is not acceptable.

If in doubt don't use any such software. In general, it appears that, as of 2025, the free version of Grammarly is fine to use, but the pro version is not.

A.4 Example of Longtable

Ticket Type ID	Description
300	Feeder Ticket - Child
301	Feeder Ticket - Adult
310	10-Journey Feeder - Adult
317	Airlink Adult Airport-Busarus
318	Airlink Child Airport-Busarus
319	Airlink Child Airport-Heuston
320	Airlink Adult Airport-Heuston
333	Adult Single Feeder
365	Child Bus/Rail Short Hop - Day
366	Adult Bus/Rail Short Hop - Day
367	Family Bus/Rail Short Hop - Day
369	4 Day Explorer
410	Weekly Adult Short Hop Bus/Rail
430	Weekly Adult Medium Hop Bus/Rail
431	Weekly Adult Long Hop Bus/Rail
432	Weekly Adult Giant Hop Bus/Rail
433	Monthly Adult Short Hop Bus/Rail
455	Monthly Adult Long Hop Bus/Rail
456	Monthly Adult Giant Hop Bus/Rail
457	Monthly Student Short Hop Bus/Rail
458	Annual Bus/Rail
478	Annual All CIE Services
479	Annual CIE Pensioner Bus/Rail
480	Monthly CIE Pensioner Bus/Rail
493	Foreign Student - 1 Week
494	Foreign Student - 2 Week
495	Foreign Student - 3 Week
496	Foreign Student - 4 Week
497	CYC Group
600	Adult Cash Fare
608	Nitelink (Maynouth/Celbridge)
609	Nitelink (Maynouth/Celbridge)
610	Child Cash Fare
620	Schoolchild Cash Fare
625	Adult (formerly Shopper)
630	Adult 10-Journey (3 Stages)
631	Adult 10-Journey (7 Stages)
632	Adult 10-Journey (12 Stages)
633	Adult 10-Journey (23 Stages)
634	Adult 10-Journey (23+ Stages)
640	Adult 2-Journey (3 Stages)
641	Adult 2-Journey (7 Stages)
642	Adult 2-Journey (12 Stages)
643	Adult 2-Journey (23 Stages)
644	Adult 2-Journey (23+ Stages)
650	Schoolchild 10-Journey
651	Scholar 10-Journey
652	Schoolchild 2-Journey
653	Scholar 2-Journey
657	Transfer 90 (or Passenger Change)
658	Adult Single Heuston-CC
660	Adult One Day Travelwide
661	Child One Day Travelwide
662	Family One Day Travelwide
665	Rambler (3 Day Bus only)
670	Weekly Adult Bus

Ticket Type ID	Description
671	Weekly Adult Cityzone
690	Weekly Student Travelwide
691	Weekly Student Cityzone
705	Monthly Adult Citizone (AerLingus.)
710	Monthly Adult Travelwide
730	Annual Adult Travelwide
760	Annual Staff Bus
790	School Pass
791	OAP Pass
800	City Tour - Adult
801	City Tour - Family
802	City Tour - Child
898	10 - Journey Test Ticket

Glossary

ChatGPT An AI language model developed by OpenAI that can understand and generate human-like text based on prompts

Chat-GPT See ChatGPT

Chat GPT See ChatGPT

GPT-4o A multimodal Large Language Model developed by OpenAI that processes text, vision, and audio inputs natively and in real time

DeepSeek-R1 An open-source Large Language Model developed by DeepSeek, designed for general-purpose reasoning and code understanding

Retrieval-Augmented Generation A technique that combines information retrieval with language generation to improve the accuracy and relevance of AI-generated responses

RAG approach See Retrieval-Augmented Generation

RAG method See Retrieval-Augmented Generation

Large Language Model A type of AI trained on vast text data to understand, generate, and manipulate human language

language model See Large Language Model

LLMs See Large Language Model

Artificial Intelligence The simulation of human intelligence in machines designed to think, learn, and solve problems

machine intelligence See Artificial Intelligence

AI systems See Artificial Intelligence

Machine Learning A subset of artificial intelligence that enables systems to learn and improve from experience without being explicitly programmed

ML system See Machine Learning

ML model See Machine Learning

Malware Malicious software designed to disrupt, damage, or gain unauthorized access to computer systems

malicious software See Malware

malware sample See Malware

Decompilation The process of translating low-level machine code or bytecode back into a higher-level, human-readable source code

decompile See Decompilation

decompiles See Decompilation

Decompiler See Decompilation

PostgreSQL An advanced, open-source relational database management system known for its extensibility and SQL compliance

Postgres See PostgreSQL

pgvector An extension for PostgreSQL that provides support for vector similarity search, enabling efficient handling of embeddings and machine learning workloads

pg-vector See pgvector

PEFrame An open-source tool used for static analysis of Portable Executable (PE) files to detect malware and extract useful information

PE frame See PEFrame

VirusTotal A free online service that analyses files and URLs for viruses, worms, trojans, and other kinds of malicious content using multiple antivirus engines

MalwareBazaar A platform for sharing and downloading malware samples, aimed at aiding cybersecurity researchers and analysts in studying malware

Haystacks An open-source AI framework designed for building end-to-end pipelines for tasks such as question answering, document retrieval, and natural language processing

Acronyms

GPT-4o Generative Pre-trained Transformer 4 Omni

DeepSeek-R1 DeepSeek Research Model 1

RAG Retrieval-Augmented Generation

LLM Large Language Model

AI Artificial Intelligence

ML Machine Learning

Bibliography

[1] H. S. Anderson and P. Roth. "EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models". In: *ArXiv e-prints* (Apr. 2018). arXiv: 1804.04637 [cs.CR].

[2] Giancarlo Cassanova, Anderies, and Alexander Agung Santoso Gunawan. "Systematic Literature Review of Artificial Intelligence in Malware Detection". In: *2022 1st International Conference on Software Engineering and Information Technology (ICoSEIT)*. 2022, pp. 18–23. DOI: 10.1109/ICoSEIT55604.2022.10029973.

[3] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. arXiv: 1810.04805 [cs.CL]. URL: https://arxiv.org/abs/1810.04805.

[4] Andrej Fedák and Jozef Štulrajter. "Fundamentals of static malware analysis: principles, methods and tools". In: *Science & Military Journal* 15.1 (2020), pp. 45–53.

[5] Mohamed Amine Ferrag et al. "Generative AI and Large Language Models for Cyber Security: All Insights You Need". In: (May 2024). DOI: 10.48550/arXiv.2405.12750.

[6] Shota Fujii and Rei Yamagishi. *Feasibility Study for Supporting Static Malware Analysis Using LLM*. 2024. arXiv: 2411.14905 [cs.CR]. URL: https://arxiv.org/abs/2411.14905.

[7] Matthew Gaber, Mohiuddin Ahmed, and Helge Janicke. "Malware Detection with Artificial Intelligence: A Systematic Literature Review". In: *ACM Computing Surveys* 56 (Dec. 2023). DOI: 10.1145/3638552.

[8] Yunfan Gao et al. "Retrieval-augmented generation for large language models: A survey". In: *arXiv preprint arXiv:2312.10997* 2.1 (2023).

[9] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle et al. Vol. 33. Curran Associates, Inc., 2020, pp. 9459–9474. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.

[10] Shivani Metta et al. *Generative AI in Cybersecurity*. 2024. arXiv: 2405.01674 [cs.CR]. URL: https://arxiv.org/abs/2405.01674.

[11] Farzad Nourmohammadzadeh Motlagh et al. *Large Language Models in Cybersecurity: State-of-the-Art*. 2024. arXiv: 2402.00891 [cs.CR]. URL: https://arxiv.org/abs/2402.00891.

[12] Bhaskar V Patil and Rahul J Jadhav. "Computer virus and antivirus software a brief review". In: *International Journal of Advances in Management and Economics* 4.2 (2014), pp. 1–4.

[13] Pranab Sahoo et al. "A systematic survey of prompt engineering in large language models: Techniques and applications". In: *arXiv preprint arXiv:2402.07927* (2024).

[14] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

[15] Miuyin Wong et al. "An Inside Look into the Practice of Malware Analysis". In: Nov. 2021, pp. 3053–3069. DOI: 10.1145/3460120.3484759.

[16] Wei Zou et al. *PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models*. 2024. arXiv: 2402.07867 [cs.CR]. URL: https://arxiv.org/abs/2402.07867.